

ALP

FICHA DE EXERCÍCIOS
ATIVIDADE LETIVA

Programação Orientada a Objetos

UNIDADE CURRICULAR

Ficha 4 – Classes e Herança

FICHA

Crie uma pasta para esta ficha (sugestão de nome: F04_Classes e Herança).

Dentro dessa pasta, crie os ficheiros `index.html` e `script.js`, e ligue-os entre si.

Todo o código JavaScript deve estar no ficheiro `script.js`.

Utilize `console.log()` para verificar resultados intermédios.

Esta ficha introduz classes em JavaScript. Os exercícios são progressivos e baseiam-se todos no mesmo cenário: o SoundBase. O ponto de partida são os objetos literais com que trabalhou nas fichas anteriores, desta vez modelados com classes. Leia cada exercício na totalidade antes de começar.

O ficheiro `soundbase-dados.js` está disponível no Moodle. Copie-o para a pasta desta ficha e ligue-o ao `index.html` antes do `script.js`. As funções `formatDuration` e `formatPlays` já estão incluídas nesse ficheiro; não precisa de as redefinir.

A Classe `Track`

- 1) Crie uma classe chamada `Track` com um constructor que recebe os parâmetros `title`, `artist`, `duration` e `genre`. Adicione ainda as propriedades `plays` (valor inicial 0) e `liked` (valor inicial `false`).
- 2) Crie 3 instâncias de `Track` com `new Track(...)` e use `console.log()` para confirmar que as propriedades foram inicializadas corretamente.
- 3) Adicione os seguintes métodos à classe `Track`:
 - `play()` – incrementa `plays` em 1 e imprime "Now playing: Blinding Lights by The Weeknd (plays: 1)".
 - `like()` – alterna `liked` entre `true` e `false` e imprime "Blinding Lights liked!" ou "Blinding Lights unliked!".
 - `getInfo()` – retorna uma string com o formato: "Blinding Lights – The Weeknd | 3:20 | Pop | 0 plays". Use `formatDuration` e `formatPlays`.
 - Teste os três métodos com uma das instâncias.

- 4) Torne `plays` e `liked` propriedades privadas com o prefixo `#`. Atualize os métodos que as utilizam. Adicione os getters `get plays()` e `get liked()` para permitir leitura externa.
 - Confirme que tentar aceder a `track.#plays` fora da classe lança um erro.

Nota: Em JavaScript, não existe a keyword `protected`. O equivalente convencional é prefixar propriedades com `_` (ex: `_property`) como sinal de “uso interno” – mas trata-se apenas de uma convenção; a linguagem não impõe nenhuma restrição de acesso. Para encapsulamento real, use `#`.
- 5) Adicione um setter `set plays(value)`: se `value` for negativo, imprima "Erro: o número de plays não pode ser negativo." e não atualize o valor. Teste com um valor negativo e com um valor válido.

Métodos Estáticos

- 6) Adicione à classe `Track` o método estático `Track.compare(a, b)` que recebe dois objetos `Track` e retorna o que tem mais plays. Se os valores forem iguais, retorne `null`. Aceda a `plays` pelos getters. Teste com dois pares de instâncias.
- 7) Adicione à classe `Track` o método estático `Track.fromObject(obj)` que recebe um objeto literal com as propriedades `title`, `artist`, `duration` e `genre`, e retorna uma nova instância de `Track`. Use-o para converter entradas do catálogo:
 - `const t = Track.fromObject(catalogue[0]);`
 - Confirme com `console.log(t.getInfo())`.

A Classe Playlist

- 8) Crie uma classe `Playlist` com um constructor que recebe `name` e `createdBy`. Adicione uma propriedade privada `#tracks`, inicializada como array vazio.
- 9) Adicione os seguintes métodos à classe `Playlist`:
 - `add(track)` – adiciona o objeto `Track` ao array `#tracks` e imprime "Blinding Lights added to Late Night Vibes."
 - `remove(title)` – remove do array a faixa cujo `title` corresponde ao argumento. Imprima "Blinding Lights removed." Ou "Track not found."
 - `getTotalDuration()` – retorna a duração total formatada (ex: "Total: 14:20").
 - `getMostPlayed()` – retorna a instância `Track` com mais plays.
 - `printAll()` – imprime o cabeçalho "Late Night Vibes – by mina_1e6" seguido de cada faixa com `getInfo()`.
- 10) Adicione um getter `get size()` que retorna o número de faixas na playlist. Teste todos os métodos com pelo menos 3 instâncias de `Track`.

Herança

- 11) Crie a classe `PodcastEpisode` que estende `Track`. O constructor deve receber `title`, `host`, `duration` e `episode` (número do episódio). Use `super()` para inicializar a classe pai – passe `host` como `artist` e `"Podcast"` como `genre`. Adicione a propriedade `episode` ao constructor.

Nota: `super()` deve ser a primeira instrução dentro do constructor de uma subclasse. Tentar usar `this` antes de chamar `super()` lança um erro.

- 12) Faça override do método `getInfo()` para incluir o número de episódio. Exemplo de output: `"Ep. 12 – Deep Dive into AI | hosted by Lex Fridman | 1:22:45"`.
 - Pode usar `super.getInfo()` se quiser reutilizar parte da lógica existente.
- 13) Crie 2 instâncias de `PodcastEpisode` e adicione-as a uma `Playlist`. Chame `printAll()` e confirme que o `getInfo()` sobreescrito é chamado corretamente para os episódios.

Desafios

- 14) Adicione à classe `Track` um campo estático privado `#count = 0` e um campo de instância privado `#id`. No constructor, incremente `#count` e atribua o valor atual a `#id`. Adicione o getter `get_id()` e o getter estático `static get count()`. Confirme que cada instância criada tem um `id` único e crescente.
- 15) Adicione à classe `Playlist` o método `getByGenre(genre)` que retorna um array com as faixas do género indicado. Imprima cada resultado com `getInfo()` ou a mensagem `"No tracks found for genre: Jazz"`.